x86-Arch-Notes

CPU has Arithmetic Logic Unit, Registers and Control Unit.
ALU connects to both CU and Regs. This contraption is called Von-neumann architecture for some reason.
CU handles the instructions, keeping the Instruction pointer (EIP in 32bit and RIP in 64bit) which knows which instruction to execute next.
ALU is the one obtaining instructions from mem, executing it and then storing the result in either Mem or Regs.
Regs are just fast and easily accessible storage for CPU.

Register Types

- **Instruction Pointer** : also called the Program counter, contains the addr of next instruction. In 32bit called Extended Instruction Pointer and in 64bit systems called Reg-Instruction Pointer.
- **Gen-Purpose Reg** : All 32-bit regs in x86 systems and 64-bit in x64 systems. [Here the 'E' prefix is for 32-bit and 'R' for 64-bit. Suffix 'X' is usually used for standard-reg and 'P' for pointer-reg usually and 'I' for index-reg].
  Has the following in it.->
  - ***Accumulator Reg*** : often stores the **results of Arithmetic Operations**. 32-bit has `EAX`, 64-bit has `RAX`. Inside these, last 16-bits can be addressed using `AX` and within it, you can address as as two 8-bits, `AL` for lower and `AH` for higher respectively.
  - ***Base Regs*** : often used to **store the base address for referencing an offset**. Same sub-categorization with 64-bit `RBX`, 32-bit `EBX`, 16-bit `BX`, 8-bit `BH` and `BL` regs.
  - ***Counter Regs*** : often **used in counting ops** such as loops etc. Same stuff with 64-`RCX`, 32-`ECX`, 16-`CX`, 8-`CH` and 8-`CL`.
  - ***Data Regs*** : often **used in multiplication/division ops**. 64-`RDX`, 32-`EDX`, 16-`DX`, 8-`DH` and 8-`DL`.
  - ***Stack Pointer*** : This reg **points at the top of the stack** and is used in conjunction with Stack Segment Reg. 64-`RSP`, 32-`ESP` and 16-`SP`. Cannot be addressed smaller than this.
  - ***Base Pointers*** : Used to **access parameters passed by the stack** and is also used in conjunction with 'Stack Segment' Reg. 64-`RBP`, 32-`EBP` and 16-`BP` (nothing smaller).
  - ***Source Index Reg*** : **Used for string ops**. Used with 'Data Segment' (DS) reg as an offset. 64-`RSI`, 32-`ESI` and 16-`SI` (nothing smaller).

    - ***Destination Index Reg*** : **Also for string ops**. Used with 'Extra Segment' (ES) reg as an offset. 64-`RDI`, 32-`EDI` and 16-`DI` (nothing smaller).
    - ***R8-R15*** : Also Gen-purpose but not present in 32-bit systems. Inherently 64-bit. Also addressable in 32,16,8-bit modes. Eg: `R8D` for lower 32-bit addressing and `R8W` for lower 16-bit addressing and R8B for lower 8-bit addressing. Here, D means 'double word' and W means 'word' and B is for Byte.
    -
- **Status Flag Reg** : Usually a status is needed for the execution. **`EFLAGS`** register for 32-bit system and **`RFLAGS`** for 64-bit. Some necessary flags are :
    - **Zero Flag** : ZF , denotes the status of the last executed instruction was 0. Eg: RAX subtracted from itself, result 0 hence ZF is set to 1.
    - **Carry Flag** : CF indicates when the last executed instruction resulted in a number too big/small for destination, setting CF to 1. Eg: `0xFFFFFFFF` added with `0x00000001` but the result is too big for a 32-bit reg.
    - **Sign Flag** : SF indicates if the result of an op is -ve OR the most significant bit it set to 1. If this is met then SF is set to 1, else 0.
    - **Trap Flag** : TF indicates if the processor is in debug-mode. When TF is set, CPU executes one instruction at a time for debugging purpose. CAN be used my malware to see if they're being run in a debugger.
- **Segment Reg** : 16-bit regs that convert flat mem space into different segments for easier addressing.
    - **Code Segment** : CS -- points to code section in mem.
    - **Data Segment** : DS -- points to program's data section in mem.
    - **Stack Segment** : SS -- points to program's stack in mem.
    - **Extra Segments** : ES, FS and GS -- point to different data sections. These and the DS regs divide the program's mem into 4 distinct data sections.

Memory Overview

By default, on load the program won't have access to full mem instead limited to his own area in mem. 4 Components -> Stack, Heap, Code & Data.

- **Code** : Contain's the program's code -- refers to the ***text*** section in a PE (portable executable) file, which includes instructions executed by the CPU. This section of mem has `chmod +x` enabled for CPU (talking about this one only).
- **Data** : const/non-var data that's initialized. Refers to a ***data*** section in a PE. Often contains Global Vars + other data not meant to change during prog's execution.
- **Heap** : aka Dynamic mem, contains vars+data created & destroyed during prog exec. When a var is created, mem is runtime allocated and freed when var is

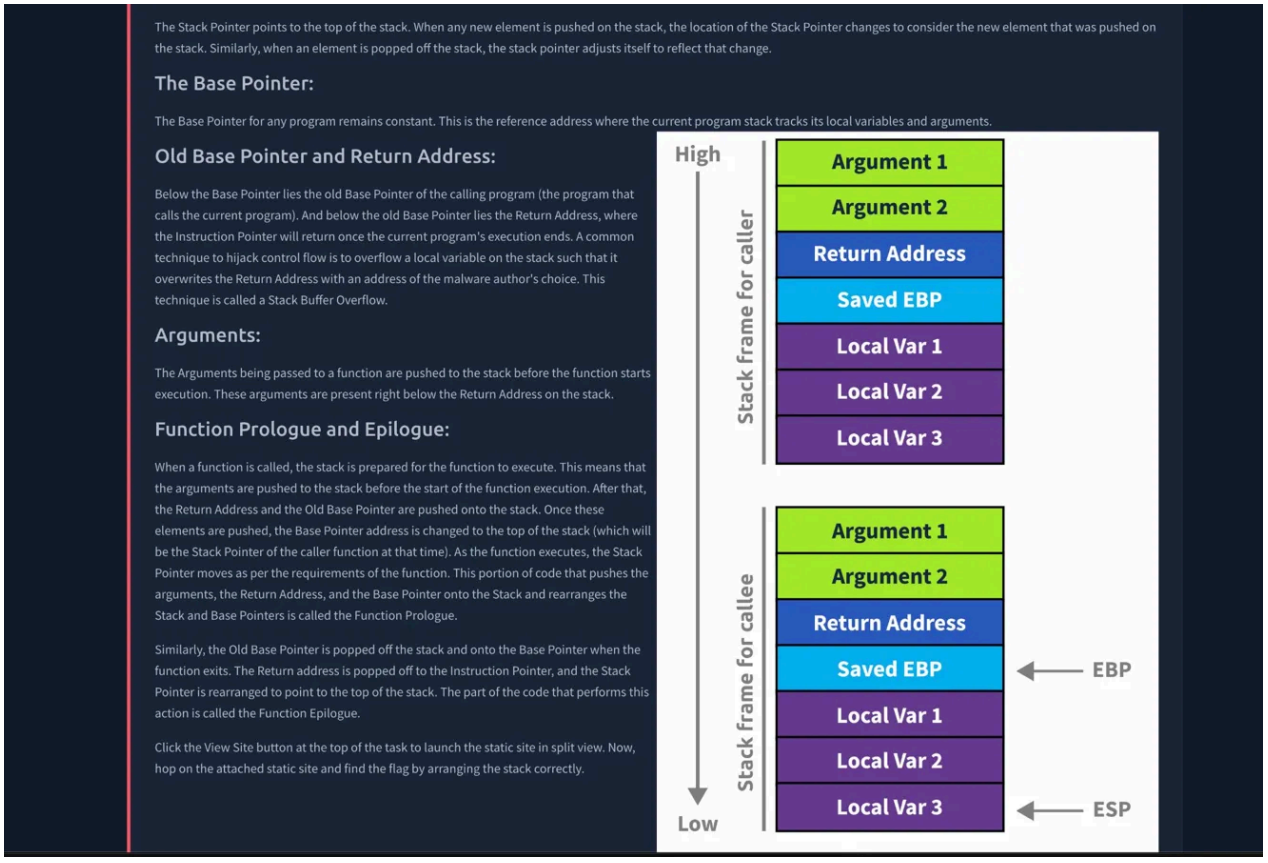
deleted.

- **Stack** : Contains local vars, args passed onto a prog & return addr of parent process that called the prog -- i.e info about prog's control flow. Since return add is related to control flow of CPU's instructions, the stack is often a malware target -- to control the flow (eg: buffer overflow).

Stack Layout

Stack is a part of the prog's mem that contains the args passed to the prog + local vars + flow control info. Stack follows LIFO based mem. CPU uses 2 regs to keep track of this - Stack Pointer (ESP/RSP) and Base Pointer (EBP/RBP).

- **Stack Pointer** : Points to the top of the stack. Moves when a new element is inserted OR older one is popped off. Like a HDD platter lol.
- **The Base Pointer** : Constant. This is the ref-addr where the current prog stack tracks its local vars+args. Acts as the navigator, telling where the args or the local-vars could be in the stack.
- **Old Base Pointer & Return Addr** : Below the Base Ptr lies the old Base Ptr of the calling prog (the one who called the current prog), and below this old Base Ptr lies the Return Addr, where the Instruction Ptr will return one the current prog's exec ends.
  [Common tech to hijack control flow is to overflow a local var on the stack such that it overwrites the return addr with an addr of malware author's choice -- called Stack Buffer Overflow].
- **Arguments** : Args being passed to a Fn are pushed to the stack before the Fn starts exec -- these are present right below the return addr on the stack.
- **Fn Prologue & Epilogue** : When a func is called, the stack is prepared for it to get exec'ed -- meaning, args are pushed to the stack before the start of Fn exec, post which the Return Addr + the old Base Ptr are pushed onto the stack. Once these are pushed, the Base Ptr addr is changed to the top of the stack (which will be the stack ptr of the caller Fn at that time).
  As the Fn execs, the Stack Ptr moves as per the reqt of the Fn. This portion of code that pushes the Args, the Return Addr'es & the Base Ptr onto the stack & rearranges the Stack and Base Ptr is called 'Fn Prologue'.
  Similarly, the Old Base Ptr is popped off the stack & onto the Base Ptr when the Fn exits. The return addr is popped off to the Instr'n Ptr & the Stack Ptr is rearranged to point to the top of the stack. The part of the code that performs this action is called 'Fn Epilogue'.

Desc :

- ESP always points to the top of the stack -- the lowest currently used address -- every `push` moves it down and every `pop` moves it up.
- EBP points to a fixed anchor for the current Fn's stack frame. Won't move while a Fn is executing -- this stability lets the CPU reliably find local vars/args (using fixed offsets like `[ebp-4]` or `[ebp+8]`).
- Stack Frame : A private-workspace for a Fn -- to store its local vars, know its args & know where to return after completion. Every Fn has it's own frame -> When Fns call each other, frames stack on top of each other in mem. Args pushed by caller are higher addrs and local vars are lower addrs. Local vars are what the ESP points too AND are the top of the stack.
- Basically a frame is prepared as and when needed. The two frames are the same things but during different times. The 'caller' thinks his frame is complete -- containing local vars, saved EBP from whoever called it, a return addr and just pushed Arg1 and 2 onto the stack to prepare for the upcoming call (to basically give it to the Fn that will be called).
- When the `CALL` instrn is exec, we jump inside the callee Fn. Callee's Fn-Prologue ran -- it pushed the caller's EBP to save it ('Saved EBP') then set EBP = ESP (moving the base pointer down to anchor the new frame), then moved ESP further down to make room for it's own local vars.
-